



Università di Brescia

*DIPARTIMENTO DI INGEGNERIA DELL'
INFORMAZIONE*

Corso di Laurea
in INGEGNERIA
INFORMATICA

PROGETTAZIONE APPLICAZIONI WEB FULL STACK

PIANIFICAZIONE APPELLI

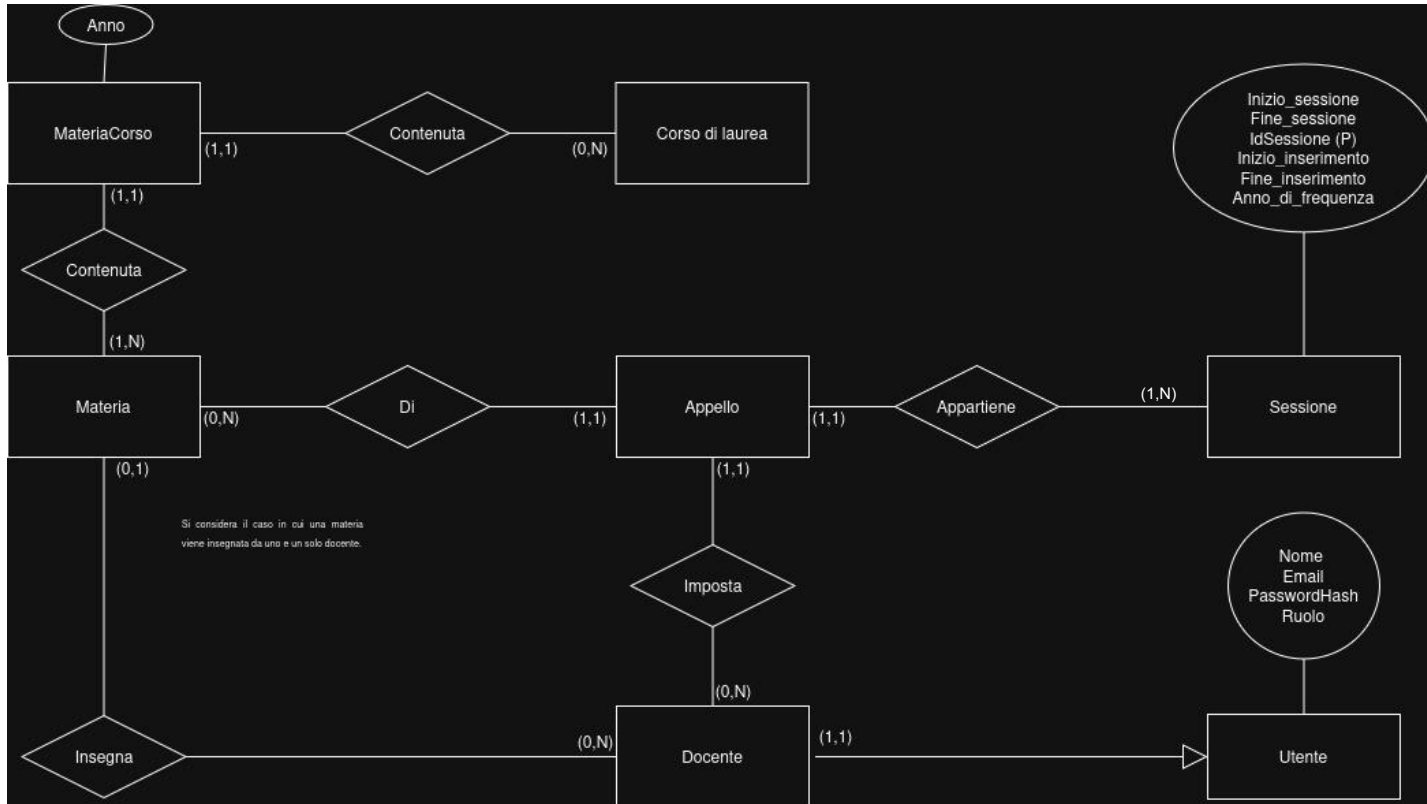
Matteo Lublanis
Singh Sahiljit

736418, 740552

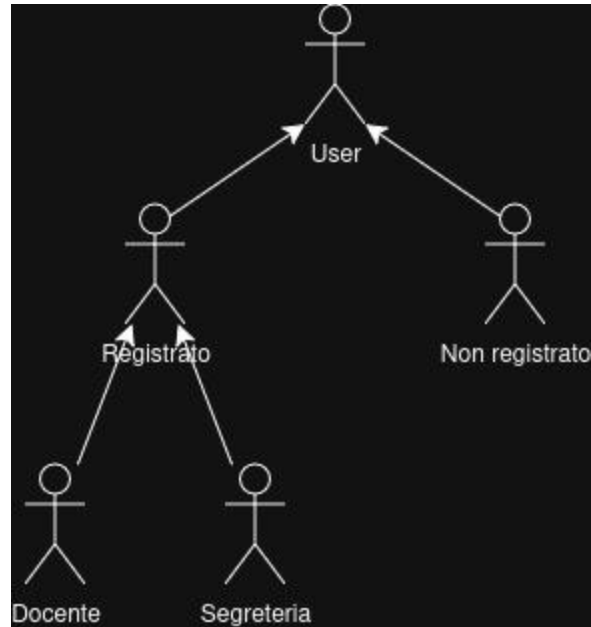
Outline

1. ER Model
2. User hierarchy
3. CRUD Matrix
4. Code structure:
 - Frontend: UI
 - Backend: controller, entities, repository, service, module, index, interfaces, DTOs
5. Hypertext model

Modello ER



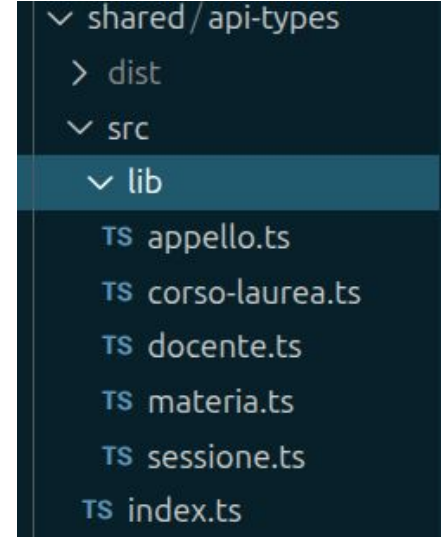
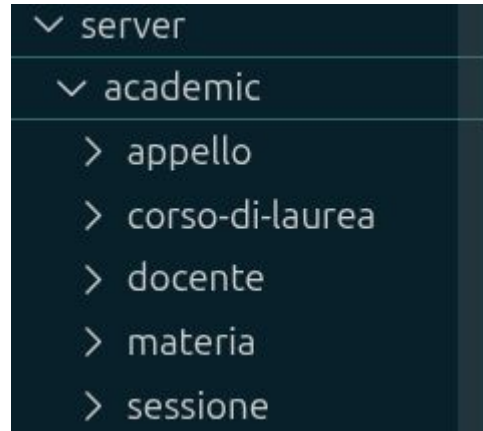
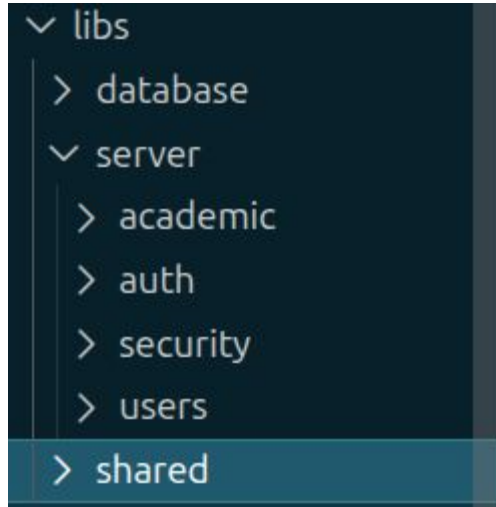
User Hierarchy



CRUD Matrix

#	Docente	Segreteria	Non registrato
CorsoDiLaurea	R	RW	-
Materia	R	RW	-
Appello	R(W)	R	-
Sessione	R	RW	-
Docente	R(W)	RW	-
Segreteria	-	R	-

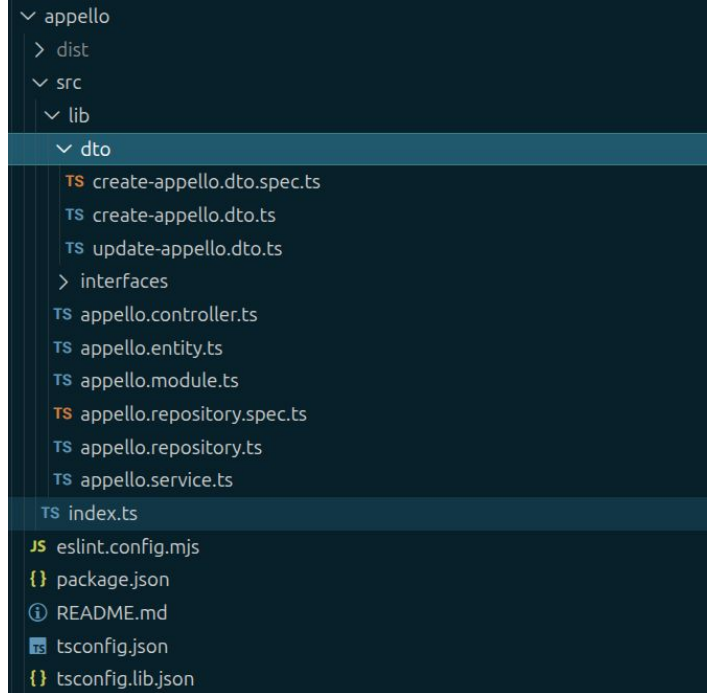
Backend



Backend

```
export interface Appello {  
  id: number;  
  data: string;  
  ora: string;  
  aula: string;  
  note?: string;  
  sessione: Sessione;  
  materia: {  
    id: number;  
    nome: string;  
    cfu: number;  
    docente?: Docente;  
    corsi: { id: number; anno: number; corso: { id: number; nome: string } }  
  };  
  docente: Docente;  
}  
  
export interface ICreateAppello {  
  data: string;  
  ora: string;  
  aula: string;  
  note?: string;  
  materiaId: number;  
  sessioneId: number;  
}
```

Backend



Backend

```
import { IsString, IsNotEmpty, IsOptional, IsInt, IsDateString } from 'class-validator';
import { ApiProperty } from '@nestjs/swagger';
import type { ICreateAppello } from '@shared/api-types';

export class CreateAppelloDto implements ICreateAppello {
  @ApiProperty({ example: '2025-06-15' })
  @IsDateString({}, { message: 'La data non è una data ISO valida' })
  data: string;

  @ApiProperty({ example: '09:30:00' })
  @IsString()
  @IsNotEmpty()
  ora: string;

  @ApiProperty({ example: 'A1' })
  @IsString()
  @IsNotEmpty()
  aula: string;

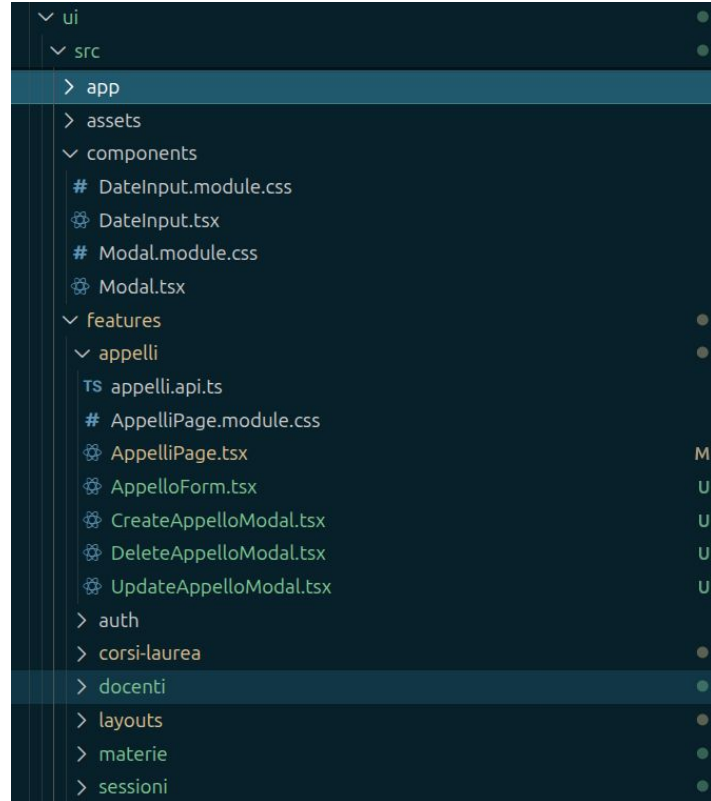
  @ApiProperty({ required: false, example: 'Portare calcolatrice' })
  @IsOptional()
  @IsString()
  note?: string;

  @ApiProperty({ example: 1 })
  @IsInt()
  materiaId: number;
```

Backend

```
@Post()
@UseGuards(RolesGuard)
@Roles(UserRole.USER)
@ApiBearerAuth()
@ApiBody({ type: CreateAppelloDto })
async create(@Body(new ValidationPipe({ transform: true, whitelist: true }))
  | | | | | data: CreateAppelloDto, @CurrentUser() user: AuthenticatedUser) {
  const docente = await this.docenteService.getByUserId(user.id);
  return this.service.create(data, docente.id);
}
```

Frontend



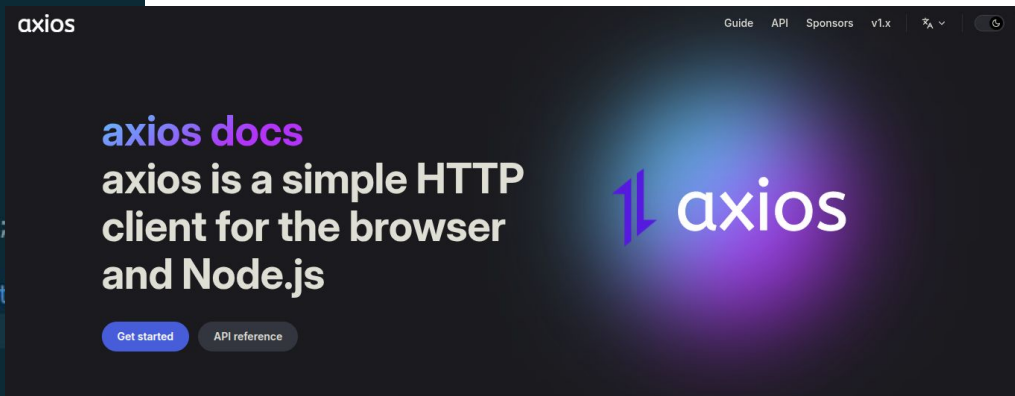
Frontend - Axios

```
import axios from 'axios';

const client = axios.create({
  |  baseURL: 'http://localhost:3333/api', //
});

client.interceptors.request.use((config) => {
  |  const token = localStorage.getItem('token');
  |  if (token) {
  |    config.headers.Authorization = `Bearer ${t
  |  }
  |  return config;
});

export default client;
```



<https://axios.rest/pages/getting-started/first-steps.html>

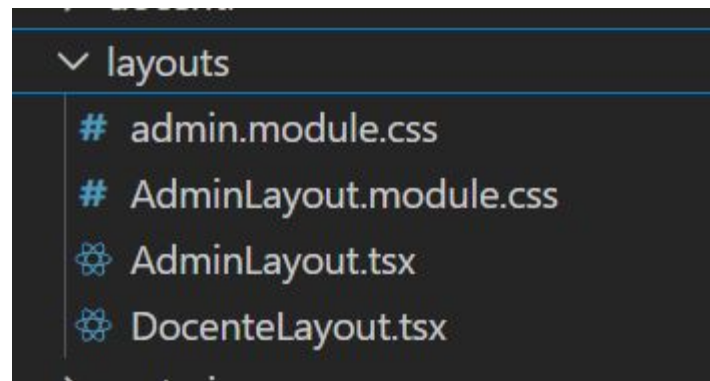
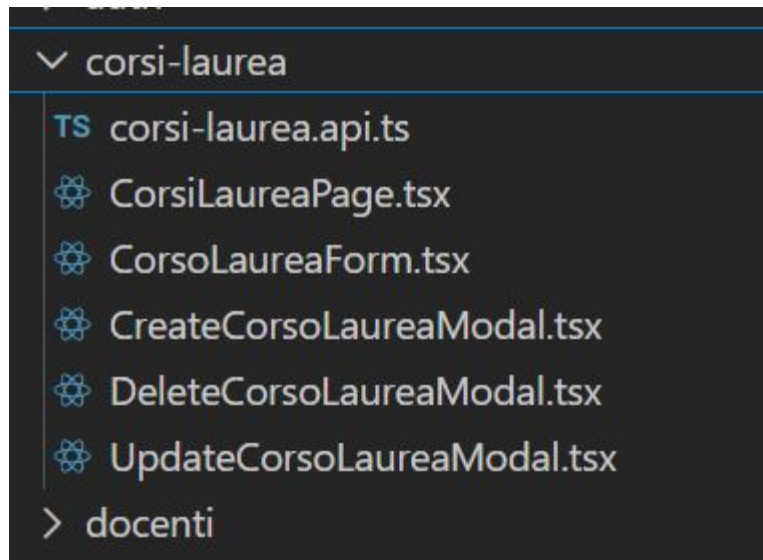
Frontend - Axios

```
export async function login(email: string, password: string) {  
  const response = await fetch(`${API_URL}/auth/login`, {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json',  
    },  
    body: JSON.stringify({ email, password }),  
  });  
  
  if (!response.ok) {  
    throw new Error('Credenziali non valide');  
  }  
  
  const data = await response.json();  
  
  localStorage.setItem('access_token', data.access_token);  
  
  return data;  
}
```

AXIOS

```
export async function login(email: string, password: string): Promise<AuthResponse> {  
  const res = await client.post<AuthResponse>('/auth/login', { email, password });  
  return res.data;  
}
```

Frontend - Struttura comune



Hypertext Model

